UNIVERSIDADE FEDERAL ALFENAS (UNIFAL)
Bacharelado em Ciência da Computação

Disciplina	Método de realização	Data de entrega
DCE797 - AEDS 3	Relatório e apresentação	12/04/2026 às 23h59

Professor
Iago Augusto de Carvalho (iago.carvalho@unifal-mg.edu.br)

Trabalho prático 2 - Pareamento de sequências de DNA

Introdução

O DNA (Ácido Desoxirribonucleico) é a molécula que carrega as instruções genéticas de todos os seres vivos, orquestrando seu desenvolvimento e funcionamento. Ele atua como um "manual de instruções"codificado, armazenando informações hereditárias — metade herdada de cada progenitor — essenciais para a produção de proteínas e características biológicas.

O DNA é formado por uma dupla hélice: uma fita dupla helicoidal, semelhante a uma escada espiralada, descoberta com contribuições de Rosalind Franklin e definida por Watson e Crick. Além disso, cada uma das "hélices"é composta por uma sequência de bases nitrogenadas: Adenina (A), Timina (T), Citosina (C) e Guanina (G). Elas se emparelham de forma específica: A se emparelha com T e C se emparelha com G.

O problema do pareamento de sequências de DNA é um pilar fundamental da bioinformática. Este problema **pode ser resolvido em tempo polinomial** buscando encontrar a melhor forma de sobrepor duas ou mais sequências de nucleotídeos (A, T, C, G) para criar regiões de dupla hélice corretas, isto é, emparelhando sempre uma base A com uma base T e uma base C com uma base G.

O objetivo deste segundo trabalho é implementar e avaliar algoritmos para realizar o pareamento de sequências de DNA.

Definições

Entrada do problema: O problema recebe como entrada duas sequências de DNA.

Sequência de DNA: Uma sequência de DNA é representada por uma string s. Esta string tem um total de n posições e pode conter unicamente os caracteres A, T, C e G. Cada caractere de uma *string* é considerada uma *base*.

Bases pareadas: Sejam s_1 e s_2 *strings* que representam duas sequências de DNA. Considera-se que duas bases $s_1[i]$ e $s_2[i]$ estejam pareadas se

- $s_1[i] = A$ e $s_2[i] = T$, ou
- $s_1[i] = T$ e $s_2[i] = A$ ou
- $s_1[i] = C$ e $s_2[i] = G$ ou
- $s_1[i] = G$ e $s_2[i] = C$.

Bases não pareadas: De forma complementar, considera-se *não pareadas* duas bases $s_1[i]$ e $s_2[i]$ que não correspondem a definição acima.

Modificações permitidas: É permitido modificar qualquer uma das *strings* inserindo lacunas (gaps), representados por um símbolo $-$. Estas lacunas podem ser inseridas em qualquer posição da *string*. Esta operação é gratuita, mas terá implicações na pontuação de pareamento de sequências de DNA.

Pontuação de pareamento entre sequências de DNA: Duas sequências de DNA recebem uma **nota de pareamento** de acordo com o pareamento de suas bases. Um par de bases pareado recebe uma pontuação de 2 (dois pontos). Já um par de bases não pareados recebe uma pontuação de -1 (menos um ponto). Além disso, uma base pareada com uma lacuna recebe uma pontuação de -2 (menos dois pontos).

Objetivo: O objetivo é alinhar ambas as *strings* introduzindo lacunas como forma de maximizar a pontuação de pareamento entre ambas as sequências de DNA.

Exemplo 1

Sejam s_1 = `ACACACTA` e s_2 = `TCGTGTGT`. Considere que a cor azul represente um par de bases pareados, a cor vermelha represente um par de bases não pareados e a cor laranja representa o pareamento de uma base com uma lacuna. Temos que a *pontuação de pareamento* entre s_1 e $_2$ é -2.

```
ACACACTA
TCGTGTGT
```

Podemos inserir duas lacunas - uma na *string* s_1 e outra na string s_2 - para aumentar a pontuação de pareamento destas *strings*. Assim, podemos reescrever s_1 = `A-CACACTA` e s_2 =`TCGTGTG-T`. A pontuação de pareamento destas novas strings será 10.

```
A-CACACTA
TCGTGTG-T
```

Exemplo 2

Sejam s_1 = `CATCG` e s_2 = `TAGCC`. Temos que a *pontuação de pareamento* entre s_1 e s_2 é -2.

```
CATCG
TAGCC
```

Neste problema também podemos inserir duas lacunas - uma em cada string - para aumentar a nota de pareamento de ambas. Podemos reescrever s_1 = `CATCG-` e s_2 = `-TAGCC`. A pontuação de pareamento destas novas *strings* será 4.

```
CATCG-
-TAGCC
```

Definição dos grupos: Vamos trabalhar com grupos de **4** ou **5** alunos. Grupos com um número diferente de alunos serão **penalizados** em 10% de sua nota final. Cada aluno deverá acessar o Moodle e escolher seu grupo neste link. Vocês podem manter os grupos do trabalho anterior ou definir novos grupos para este trabalho.

O que deve ser desenvolvido: Cada grupo deverá desenvolver dois algoritmos para a resolução do problema do pareamento de sequências de DNA. Um dos algoritmos deverá ser uma **programação dinâmica**, enquanto o segundo algoritmo deverá ser um **algoritmo guloso**. Os algoritmos **necessariamente** deverão seguir o modelo em anexo a este trabalho. Somente os arquivos **algoritmos.c** e **algoritmos.h** podem ser modificados para a execução deste trabalho.

Os grupos deverão realizar experimentos com um conjunto de instâncias em anexo a este trabalho e deverão reportar, para cada algoritmo, seu tempo de execução e sua *pontuação de pareamento*. Note que estes calculos já estão implementados no algoritmo base dado.

Além dos algoritmos, cada grupo deverá desenvolver um relatório em formato *.pdf*. Este relatório poderá ser desenvolvido em qualquer editor de texto que o grupo consiga trabalhar e deverá conter, no mínimo:

- Introdução: Explicação teórica do problema de alinhamento.
- Metodologia: Descrição de como cada paradigma foi implementado.
- **Gráficos**: Um gráfico relacionando o tamanho da Entrada vs. tempo de execução. Além disso, também deverá ser construído um gráfico relacionando o tamanho da entrada vs. consumo de memória.
- **Tabela de pontuação**: Uma tabela contendo as pontuações de pareamento de ambos os algoritmos para cada instância. O objetivo é verificar se o algoritmo guloso encontrou a mesma pontuação que o algoritmo de programação dinâmica.
- **Análise Crítica**: Discussão sobre o "trade-off"entre tempo de execução e otimalidade dos resultados.
- **Divisão de Tarefas**: Quem fez o quê no grupo.

Além disso, os grupos também deverão montar uma apresentação de slides (também em formato *.pdf*) para apresentação em sala de aula nos dias 13/04 e 14/04, sendo que a apresentação deverá durar entre 7 e 12 minutos.

Método de entrega: Tanto o código como ss dois arquivos *.pdf* deverão ser entregues no Moodle da disciplina até às 23h59 do dia 12/04.

Método de avaliação: O código corresponderá por 30% da nota total. Já a apresentação corresponderá por 40% da nota total. De forma complementar, o outro documento *.pdf* corresponderá pelos 30% restantes da nota.

Na apresentação, serão avaliados:

- Adequação ao tempo
- Postura dos apresentadores
- Assertividade na fala
- Corretude da prova apresentada
- Uso correto da língua portuguesa
- Qualidade dos slides

No documento *.pdf* com a descrição da prova, serão avaliados:

- Uso correto da língua portuguesa
- Corretude da prova apresentada
- Qualidade geral da apresentação da prova; Novamente, o uso de figuras e imagens é encorajado
- Adequação ao template disponibilizado

No código serão avaliados:

- A qualidade e clareza do código
- Comentários explicativos
- Correspondência entre os algoritmos implementados e o que foi apresentado em sala (incluindo o texto do relatório)